

MINI PROJECT

DESIGN AND ANALYSIS OF ALGORITHMS

Mini project: MATRIX CHAIN OPTIMIZER USING DYNAMIC PROGRAMMING

Name: THEJASWINEE V S

Register Number: 711525BCS171

Class: COMPUTER SCIENCE AND ENGINEERING II (CSE-B)

Subject: Design and Analysis of Algorithms

Date of Submission: 13/10/2026

MATRIX CHAIN OPTIMIZER USING DYNAMIC PROGRAMMING

1. Abstract

The **Matrix Chain Optimizer** is a Dynamic Programming-based application designed to determine the most efficient order for multiplying a sequence of matrices.

Matrix multiplication is associative, which means that matrices can be grouped in different ways without changing the final result. However, different multiplication orders can require significantly different numbers of scalar multiplications. Therefore, selecting an appropriate multiplication order can reduce computational cost.

This project accepts the dimensions of a sequence of compatible matrices and uses the **Matrix Chain Multiplication Dynamic Programming algorithm** to determine the minimum number of scalar multiplications required. It also identifies the corresponding optimal parenthesization.

The project demonstrates the practical application of **Dynamic Programming, optimization, recursion, arrays, and algorithm complexity analysis**.

2. Introduction

Matrix multiplication is widely used in areas such as:

- Computer graphics
- Machine learning
- Scientific computing
- Image processing
- Computer vision
- Engineering simulations

Consider three matrices:

$$A_1 \times A_2 \times A_3$$

There are multiple ways to perform the multiplication:

$$(A_1 \times A_2) \times A_3$$

or

$$A_1 \times (A_2 \times A_3)$$

Although both produce the same final matrix, their computational costs may be different.

The purpose of this project is to find the multiplication order that requires the **minimum number of scalar multiplications**.

3. Problem Statement

When a sequence of matrices is multiplied, the order in which the matrices are grouped can greatly affect the computational cost.

For a given sequence of compatible matrices, manually determining the most efficient multiplication order becomes difficult as the number of matrices increases.

Therefore, there is a need for an algorithm that can automatically determine the **optimal parenthesization** while minimizing the number of scalar multiplications.

The proposed **Matrix Chain Optimizer** solves this problem using **Dynamic Programming**.

4. Objectives

The main objectives of the project are:

1. Accept the number of matrices from the user.
2. Accept the dimensions of all matrices.
3. Validate the matrix dimensions.
4. Calculate the minimum multiplication cost.
5. Determine the optimal multiplication order.
6. Display the minimum scalar multiplication count.
7. Display the optimal parenthesization.
8. Demonstrate the use of Dynamic Programming.
9. Analyze the time and space complexity of the solution.

5. Scope of the Project

The project can be used for:

- Learning Dynamic Programming.
- Understanding matrix multiplication optimization.
- Studying computational complexity.
- Demonstrating algorithmic optimization.
- Educational DSA applications.
- Comparing different matrix multiplication orders.

The project focuses on **finding the optimal order**, not on actually multiplying the matrices.

6. Basic Concept

Suppose we have:

$$A_1, A_2, A_3, \dots, A_n$$

where each matrix has compatible dimensions.

If:

$$A_i = p_{i-1} \times p_i$$

then the dimensions can be represented using an array:

$$P = [p_0, p_1, p_2, \dots, p_n]$$

For example:

$$P = [10, 30, 5, 60]$$

represents:

$$A_1 = 10 \times 30$$

$$A_2 = 30 \times 5$$

$$A_3 = 5 \times 60$$

7. Why Optimization Is Required

Consider:

$$A_1 = 10 \times 30$$

$$A_2 = 30 \times 5$$

$$A_3 = 5 \times 60$$

Option 1

$$(A_1 A_2) A_3$$

First:

$$10 \times 30 \times 5 = 1500$$

The resulting matrix is:

$$10 \times 5$$

Then:

$$10 \times 5 \times 60 = 3000$$

Total:

$$1500 + 3000 = 4500$$

Option 2

$$A_1 (A_2 A_3)$$

First:

$$30 \times 5 \times 60 = 9000$$

The resulting matrix is:

$$30 \times 60$$

Then:

$$10 \times 30 \times 60 = 18000$$

Total:

$$9000 + 18000 = 27000$$

Therefore:

Order 1 = 4500 multiplications

Order 2 = 27000 multiplications

The optimizer selects the order requiring **4500 scalar multiplications**.

8. Proposed Solution

The proposed system uses **Dynamic Programming**.

Instead of calculating the cost of every possible parenthesization repeatedly, the algorithm stores previously calculated results in a table.

The two important tables are:

Cost Table

$dp[i][j]$

Stores the minimum cost of multiplying matrices from A_i to A_j .

Split Table

$split[i][j]$

Stores the position at which the optimal multiplication is divided.

These tables allow us to calculate the optimal result efficiently.

9. Mathematical Formula

For a chain of matrices:

$$A_i A_{i+1} \dots A_j$$

we try every possible split position k .

The cost is:

$$dp[i][k] + dp[k + 1][j] + p_{i-1}p_kp_j$$

Therefore:

$$dp[i][j] = \min_{i \leq k < j} [dp[i][k] + dp[k + 1][j] + p_{i-1}p_kp_j]$$

Base condition:

$$dp[i][i] = 0$$

because multiplying a single matrix requires no multiplication.

10. Algorithm

Matrix Chain Optimization Algorithm

Step 1: Read the number of matrices.

Step 2: Read $n + 1$ dimensions.

Step 3: Create the DP cost table.

Step 4: Initialize the cost of multiplying one matrix to zero.

Step 5: Consider chains of length 2, 3, ..., n .

Step 6: For every chain, try every possible split position.

Step 7: Calculate the multiplication cost for each split.

Step 8: Store the minimum cost in the DP table.

Step 9: Store the corresponding split position.

Step 10: Recursively reconstruct the optimal parenthesization.

Step 11: Display the minimum cost and optimal order.

11. Flow of the Project

START

|

v

Enter Number of Matrices

|

v

Enter Matrix Dimensions

|

v

Validate Dimensions

```

|
v
Create DP Tables
|
v
Calculate Minimum Costs
|
v
Store Optimal Split Positions
|
v
Reconstruct Optimal Order
|
v
Display Minimum Cost
|
v
Display Optimal Parenthesization
|
v
END

```

12. Complete Java Program

The following version is designed to run directly as Main.java.

```

import java.util.Scanner;

public class Main {

    static long[][] dp;

    static int[][] split;

```

```

static int[] dimensions;

* Recursively prints the optimal parenthesization.

static void printOptimalOrder(int i, int j) {
    if (i == j) {
        System.out.print("A" + i);
        return;
    }

    System.out.print("(");
    printOptimalOrder(i, split[i][j]);
    printOptimalOrder(split[i][j] + 1, j);

    System.out.print(")");
}

static long matrixChainOrder(int n) {
    dp = new long[n + 1][n + 1];
    split = new int[n + 1][n + 1];
    for (int length = 2; length <= n; length++) {
        for (int i = 1; i <= n - length + 1; i++) {
            int j = i + length - 1;
            dp[i][j] = Long.MAX_VALUE;
            for (int k = i; k < j; k++) {
                long cost = dp[i][k]
                    + dp[k + 1][j]
                    + (long) dimensions[i - 1]
                    * dimensions[k]
                    * dimensions[j]
                if (cost < dp[i][j]) {
                    dp[i][j] = cost;
                }
            }
        }
    }
}

```

```

        split[i][j] = k;
    }
}
}
}
return dp[1][n];
}

static void printCostTable(int n) {
    System.out.println("\nDP Cost Table:");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i > j) {
                System.out.print("0\t");
            } else {
                System.out.print(dp[i][j] + "\t");
            }
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.println("=====");
    System.out.println("    MATRIX CHAIN OPTIMIZER");
    System.out.println("=====");
    System.out.print("Enter number of matrices: ");
    int n = scanner.nextInt();
    if (n <= 0) {

```

```

        System.out.println("Error: Number of matrices must be positive.");
        scanner.close();
        return;
    }
    dimensions = new int[n + 1];
    System.out.println("\nEnter " + (n + 1)
        + " dimensions:");
    for (int i = 0; i <= n; i++) {
        dimensions[i] = scanner.nextInt();
        if (dimensions[i] <= 0) {
            System.out.println(
                "Error: Matrix dimensions must be positive."
            );
            scanner.close();
            return;
        }
    }
}

long minimumCost = matrixChainOrder(n);
System.out.println("\n=====");
System.out.println("RESULT");
System.out.println("=====");
System.out.println(
    "Minimum scalar multiplications: " + minimumCost
);
System.out.print("Optimal multiplication order: ");
printOptimalOrder(1, n);
System.out.println();
printCostTable(n);

```

```
        System.out.println("=====");
        scanner.close();
    }
}
```

13. Why This Code Is Reliable

There are a few important improvements compared with a basic implementation.

long is used for cost

Instead of:

```
int cost
```

the program uses:

```
long cost
```

because multiplication costs can become very large for matrices with large dimensions.

Input validation

The program checks:

```
if (n <= 0)
```

and:

```
if (dimensions[i] <= 0)
```

Therefore, invalid dimensions are rejected.

Separate responsibilities

The program has separate methods for:

```
matrixChainOrder()
```

```
printOptimalOrder()
```

```
printCostTable()
```

This makes the program easier to understand and maintain.

14. Sample Input

For the matrices:

$$A1 = 10 \times 30$$

$$A2 = 30 \times 5$$

$$A3 = 5 \times 60$$

enter:

3

10 30 5 60

15. Sample Output

MATRIX CHAIN OPTIMIZER

Enter number of matrices: 3

Enter 4 dimensions:

10 30 5 60

RESULT

Minimum scalar multiplications: 4500

Optimal multiplication order: ((A1A2)A3)

DP Cost Table:

0	1500	4500
---	------	------

0	0	9000
---	---	------

0	0	0
---	---	---

16. Explanation of the DP Table

For the example:

$$A1 = 10 \times 30$$

$$A2 = 30 \times 5$$

$$A3 = 5 \times 60$$

The DP table represents:

Matrix range	Minimum cost
A1	0
A2	0
A3	0
A1 × A2	1500
A2 × A3	9000
A1 × A2 × A3	4500

The final answer is:

$dp[1][3] = 4500$

17. Module Description

Your mini-project can be divided into **five modules**.

Module 1 — Input Module

Responsible for:

- Reading number of matrices.
- Reading dimensions.
- Validating input.

Module 2 — Dynamic Programming Module

Responsible for:

- Creating the DP table.
- Calculating minimum costs.
- Storing intermediate results.

Module 3 — Optimization Module

Responsible for:

- Testing possible split positions.
- Selecting the minimum-cost split.

Module 4 — Parenthesization Module

Responsible for:

- Reconstructing the optimal order.
- Displaying the multiplication sequence.

Module 5 — Output Module

Responsible for:

- Displaying minimum cost.
- Displaying optimal order.
- Displaying the DP table.

18. Advantages

The system provides the following advantages:

- Reduces unnecessary computational operations.
- Automatically determines the optimal multiplication order.
- Avoids repeated calculations through Dynamic Programming.
- Works for different numbers of matrices.
- Provides both cost and multiplication order.
- Demonstrates an important real-world optimization technique.

19. Limitations

The current version has some limitations:

1. It determines the multiplication order but does not actually multiply the matrices.
2. It is currently a console-based application.
3. The user must provide valid matrix dimensions.
4. It focuses on computational cost measured by scalar multiplications.

These limitations can be addressed through future enhancements.

20. Future Enhancements

You can make the project more advanced by adding:

GUI

Create a Java Swing interface where users can enter matrices visually.

Graphical DP Table

Display the DP table in a proper table format.

Cost Comparison

Allow the user to enter their own multiplication order and compare its cost with the optimized order.

Matrix Visualization

Display the dimensions and intermediate multiplication steps.

Performance Analysis

Compare:

Naive recursive approach

vs.

Dynamic Programming approach

and show execution time.

Report Generation

Generate a result report containing:

- Matrix dimensions
- Minimum cost
- Optimal order
- DP table
- Execution time

21. Time Complexity

There are three nested loops in the main DP calculation:

Length



i



k

Therefore:

$$O(n^3)$$

The algorithm requires a two-dimensional table:

$$dp[n][n]$$

Therefore:

$$O(n^2)$$

space complexity.

Final Complexity

Parameter Complexity

Time $O(n^3)$

Space $O(n^2)$

22. Technologies Used

Component	Technology
Programming Language	Java
Algorithm	Dynamic Programming
Data Structure	2D Arrays
Interface	Console

Component	Technology
IDE	VS Code / Eclipse / IntelliJ IDEA
JDK	JDK 8 or later

23. Hardware Requirements

A basic computer is sufficient.

Component	Requirement
Processor	Intel/AMD processor
RAM	4 GB or above
Storage	100 MB+ free space
Operating System	Windows/Linux/macOS

24. Software Requirements

- Java Development Kit (JDK)
- Java compiler
- VS Code / Eclipse / IntelliJ IDEA / NetBeans
- Command Prompt or Terminal

25. Test Cases

Test Case 1

Input

3

10 30 5 60

Expected result

Minimum scalar multiplications: 4500

Optimal multiplication order: ((A1A2)A3)

Test Case 2

Use:

4

40 20 30 10 30

The program should calculate the minimum cost and corresponding parenthesization automatically.

Test Case 3 — Single Matrix

1

10 20

Expected:

Minimum scalar multiplications: 0

Optimal multiplication order: A1

This is correct because there is no multiplication to perform.

Test Case 4 — Invalid Number of Matrices

0

Expected:

Error: Number of matrices must be positive.

Test Case 5 — Invalid Dimension

3

10 30 -5 60

Expected:

Error: Matrix dimensions must be positive.

26. Result

The **Matrix Chain Optimizer** successfully determines the optimal order for multiplying a sequence of matrices using Dynamic Programming.

For the sample matrices:

10×30

30×5

5×60

the algorithm determines:

Optimal Order = ((A1A2)A3)

Minimum Cost = 4500 scalar multiplications

Thus, the project demonstrates how Dynamic Programming can be used to solve an optimization problem efficiently.

27. Conclusion

The **Matrix Chain Optimizer Using Dynamic Programming** provides an efficient solution for finding the optimal multiplication order of a sequence of matrices.

Although matrix multiplication is associative, different parenthesizations can have substantially different computational costs. The Dynamic Programming approach avoids repeatedly solving the same subproblems and stores intermediate results to efficiently determine the minimum cost.

The project provides practical understanding of **Dynamic Programming, optimal substructure, overlapping subproblems, recursion, arrays, optimization, and complexity analysis**.